

Informe del Entregable 1

CISTR

Basado en la implementacion del repositorio

5 de mayo de 2026

Índice

1. Introduccion y contexto del sistema	2
2. Descomposicion en tareas: criterio de diseno	2
2.1. Eleccion de prioridades	2
3. Sincronizacion: uso exclusivo de colas	3
3.1. Justificación del uso de colas	3
3.2. Inventario de colas y su semantica	3
4. Diseno de la ISR y antirrebote	3
5. Seccion critica y proteccion del contador global	4
6. Gestion de la saturacion en la unidad de preparacion	5
7. Conclusion	5

1. Introduccion y contexto del sistema

Este informe explica las decisiones de diseno tomadas durante el desarrollo del sistema de control para la planta de produccion de cemento. El firmware corre sobre un ESP32 con ESP-IDF y FreeRTOS, todo contenido en un unico archivo fuente (`main/main.c`). La idea es justificar tecnicamente las elecciones en cuanto a tareas, sincronizacion y primitivas de tiempo real, no solo describir lo que hace el codigo.

El codigo fuente completo esta disponible en el siguiente repositorio:

https://github.com/pnavarro3/E1_CISTR

2. Descomposicion en tareas: criterio de diseno

Cuando hay multiples flujos de trabajo concurrentes, la forma de no volverse loco es seguir un criterio claro de descomposicion. Aqui se eligio **una tarea por recurso activo o por etapa del pipeline**: cada tarea tiene un unico proposito, se bloquea cuando no tiene trabajo y no pisa a las demas.

La Tabla 1 recoge las 9 tareas creadas en `app_main` y la razon de cada una.

Cuadro 1: Tareas FreeRTOS y justificacion de su existencia.

Tarea	Instancias	Justificacion de diseno
<code>tarea_carga_material</code>	3	Una por material (Arena, Agua, Cemento). El mismo codigo parametrizado con <code>i</code> cubre los tres casos, cada uno gestionando su receptaculo de forma independiente.
<code>tarea_preparacion</code>	1	Unico consumidor de la cola de materiales disponibles. Con un solo hilo no hay riesgo de que dos packs se generen a partir del mismo material.
<code>tarea_procesamiento</code>	1	Unico lector de la cola de packs. Se encarga de enrutar cada pack a la estacion correcta; al ser el unico lector no necesita sincronizacion adicional.
<code>tarea_estacion</code>	3	Una por estacion de mezcla, operando en paralelo. Cada una bloquea su propia cola sin coordinarse con las otras, lo que es exactamente la Opcion Avanzada I.
<code>tarea_panel_estado</code>	1	Imprime el estado del sistema cada 2 s. Tenerla separada evita que su logica de lectura interfiera con el resto del pipeline.

2.1. Eleccion de prioridades

Todas las tareas se crean con la misma prioridad (nivel 1). Funciona porque ninguna tiene un *deadline* tan estricto que necesite expulsar a otra: los tiempos de ciclo del sistema (10 s de mezcla, 2 s de panel) son ordenes de magnitud mayores que el quantum del planificador. Si en el futuro se quisiera garantizar una latencia de respuesta baja en los botones, lo logico seria subir `tarea_carga_material` a una prioridad mayor.

3. Sincronizacion: uso exclusivo de colas

3.1. Justificación del uso de colas

FreeRTOS ofrece semaforos binarios, mutexes, semaforos contadores y colas. La decision aqui fue **usar solo colas** para toda la comunicacion entre tareas e ISR. Hay dos razones concretas:

1. **Las colas transportan datos y sincronizan a la vez.** Un semaforo avisa de que algo paso, pero no dice que. En este sistema la ISR necesita notificar una pulsacion y la tarea de carga necesita saber de que material se trata. Con una cola de capacidad 1 y elemento `uint8_t` se resuelven las dos cosas con una sola primitiva.
2. **La capacidad de la cola es la restriccion del sistema.** Un receptaculo de capacidad 1 es directamente `xQueueCreate(1, sizeof(int))`. No hay que escribir logica adicional para ese limite: la cola lo impone sola.

Un semaforo de espacio libre habria funcionado para controlar la capacidad, pero habria necesitado una segunda primitiva para pasar el ID del material a preparacion. Las colas evitan esa complejidad extra.

3.2. Inventario de colas y su semantica

Cuadro 2: Colas del sistema y su semantica de sincronizacion.

Cola	Cap.	Semantica y justificacion
<code>cola_pulsaciones[i]</code>	1	ISR $\rightarrow$ <code>tarea_carga_material[i]</code> . Capacidad 1: si la tarea aun no proceso el evento anterior, la siguiente pulsacion se descarta (el antirrebote ya la filtra antes de llegar aqui).
<code>cola_receptaculos[i]</code>	1	Representa el receptaculo fisico. Lleno cuando tiene un elemento, vacio cuando no. Un <code>xQueueSend(..., 0)</code> falla de inmediato si ya esta lleno, implementando la politica de descarte con aviso por log.
<code>cola_materiales_disponibles</code>	3	<code>tarea_carga_material</code> $\rightarrow$ <code>tarea_preparacion</code> . Capacidad 3: como mucho un material de cada tipo puede estar esperando al mismo tiempo. Hace de buffer entre carga y preparacion.
<code>cola_packs_preparados</code>	3	<code>tarea_preparacion</code> $\rightarrow$ <code>tarea_procesamiento</code> . Capacidad 3. Al llenarse, preparacion entra en espera activa y lanza un <code>ESP_LOGW</code> .
<code>cola_estacion[i]</code>	3	<code>tarea_procesamiento</code> $\rightarrow$ <code>tarea_estacion[i]</code> . Capacidad 3 para absorber rafagas de packs hacia la misma estacion mientras esta ocupada procesando otro.

4. Diseno de la ISR y antirrebote

La ISR implementada sigue exactamente ese patron:

```

1 static void isr_boton(void *arg)
2 {
3     int i = *(int *)arg;
4     BaseType_t tarea_prioritaria_despertada = pdFALSE;
5     TickType_t tick_actual = xTaskGetTickCountFromISR();
6     uint8_t evento = 1;
7
8     if (ultimo_evento_boton[i] != 0 &&
9         (tick_actual - ultimo_evento_boton[i]) < TIEMPO_ANTIRREBOTE) {
10         return; // descarta rebote
11     }
12
13     ultimo_evento_boton[i] = tick_actual;
14     xQueueSendFromISR(cola_pulsaciones[i], &evento,
15                     &tarea_prioritaria_despertada);
16     if (tarea_prioritaria_despertada) {
17         portYIELD_FROM_ISR();
18     }
19 }

```

Vale la pena revisar cada decision puntual:

- **Antirrebote en la ISR, no en la tarea.** El filtro de 1000 ms se aplica comparando *ticks* del sistema dentro de la propia ISR. Asi se cortan los eventos espurios antes de que lleguen a la cola y la tarea de carga no tiene que lidiar con ellos.
- **xQueueSendFromISR en lugar de xQueueSend.** Las versiones **FromISR** son las unicas que pueden llamarse desde una interrupcion porque no bloquean el planificador. Usar la version normal desde una ISR corrompe el estado interno de FreeRTOS.
- **portYIELD_FROM_ISR.** Si al insertar en la cola se desbloquea una tarea de mayor prioridad, se pide un cambio de contexto inmediato al salir. Con todas las tareas al mismo nivel no hay efecto practico aqui, pero es la forma correcta de escribirlo.
- **Variable volatile.** El arreglo `ultimo_evento_boton[]` se declara **volatile** para que el compilador no optimice su acceso desde la ISR y desde las tareas.

5. Seccion critica y proteccion del contador global

La variable `total_mezclas` es compartida por tres tareas de estacion que corren en paralelo. Para protegerla se usa una **seccion critica de FreeRTOS** con `portMUX_TYPE`:

```

1 portENTER_CRITICAL(&mux_mezclas);
2 total_mezclas++;
3 total_local = total_mezclas;
4 portEXIT_CRITICAL(&mux_mezclas);

```

Un mutex tiene sentido cuando la seccion critica puede tardar y conviene que el planificador ceda el procesador mientras se espera. Aqui la region es de una sola instruccion (incremento y lectura de un entero), asi que la espera activa de la seccion critica es mas rapida: sin latencia de planificacion y con tiempo de ejecucion constante.

6. Gestion de la saturacion en la unidad de preparacion

Cuando la cola de packs preparados llega a 3/3, `tarea_preparacion` no puede enviar mas y tiene que esperar. El codigo lo resuelve con un **bucle de sondeo activo** de 100 ms:

```
1 if (uxQueueSpacesAvailable(cola_packs_preparados) == 0) {  
2     ESP_LOGW(TAG, "Preparacion_llena_(3/3)...");  
3     while (uxQueueSpacesAvailable(cola_packs_preparados) == 0) {  
4         vTaskDelay(pdMS_TO_TICKS(100));  
5     }  
6 }
```

Lo ideal seria bloquearse en `xQueueSend(..., portMAX_DELAY)` y ceder el procesador hasta que haya espacio. El problema es que `tarea_preparacion` ya consumio un mensaje de `cola_materiales_disponibles` antes de llegar aqui, y no puede devolverlo. El sondeo con `vTaskDelay(100 ms)` es la salida practica: la latencia maxima de 100 ms es insignificante frente a los 10 s de ciclo de mezcla.

En el resto del sistema el bloqueo es siempre `portMAX_DELAY`, con lo que las tareas no consumen CPU mientras no tienen trabajo:

```
1 xQueueReceive(cola_pulsaciones[i], &evento, portMAX_DELAY);  
2 xQueueReceive(cola_packs_preparados,  
3             &codigo_pack, portMAX_DELAY);  
4 xQueueReceive(cola_estacion[estacion],  
5             &codigo_pack, portMAX_DELAY);
```

7. Conclusion

El sistema esta construido sobre un conjunto de decisiones tecnicas coherentes entre si:

- **Una tarea, una responsabilidad:** simplifica el razonamiento temporal y hace que los fallos sean mucho mas faciles de localizar.
- **Solo colas para sincronizar:** la capacidad configurable modela directamente las restricciones del problema y el bloqueo en `portMAX_DELAY` elimina la espera activa en casi todo el sistema.
- **ISR minima:** filtra rebotes y encola un evento, nada mas. Toda la logica real ocurre en tareas.
- **Seccion critica de espera activa para el contador:** la region es de una instruccion, por lo que la espera activa es mas rapida y sencilla que un mutex.
- **Tres estaciones en paralelo:** elimina el cuello de botella de la exclusion mutua global y maximiza el rendimiento de la planta, cumpliendo la Opcion Avanzada I.